

Project-Ticket: Complete Deep Audit & Architecture Documentation

Scope: `D:\Project-ticket\Project-Ticket` · **Date:** 2026-08-29 · **Method:** We reviewed the source code and configuration files only. We did not run any live or destructive tests, and nothing in the codebase was changed to produce this report.

This is an independent audit pass. The repository already contains `docs/PRODUCTION_READINESS_AUDIT.md` (from 2026-08-24) and `docs/SECURITY_REMEDIATION_REPORT.pdf`. Cross-check the findings here against those two documents rather than treating this report as the very first audit ever done.

1. Executive Summary

Project-Ticket is a role-based internal tool for day-to-day operations (tickets and complaints, HR desk, leave requests, IT equipment tracking, approvals, coordinator tasks, and production render jobs). It's built as a React 18 + Vite website (a "single-page app," meaning the page never fully reloads) that talks to an Express-based backend API, which is a thin, consistent layer sitting on top of Firebase (Firestore for data, Firebase Auth for logins, Firebase Storage for documents). It's hosted as two independent Vercel projects, `project-ticket` for the website and `backend` for the API, under one shared Vercel account. There's no Docker setup and no automated deployment-checking pipeline (commonly called CI/CD).

The codebase shows clear evidence of deliberate security work done before this audit: ownership checks on the server for every complaint-editing page, a system for revoking login sessions, protection against repeated password guessing (with the offending IP address logged), a trusted-website allow-list, outbound email text that's safely escaped, and a file-upload pipeline that checks file type and size.

The gaps that do exist are structural, not scattered randomly: there's no `helmet` (a tool that adds security-related response headers, and right now there are zero of those anywhere), rate limiting (which slows down abuse) is only applied to four login-related pages, there's no minimum password rule enforced by the app itself, and two specific pages (`renderRoutes.js` , and the task-status update inside `coordinatorRoutes.js`) have no role restriction at all. Comments in the code suggest this last point is intentional, but it's still worth a second look. A file with a concerning name, `docs/login-credentials.pdf` , sits in the project folder. It's excluded from version control (confirmed not tracked), but it should still be removed or moved somewhere else regardless.

Bottom line: this is a mid-maturity internal tool with real, working access-control logic on the backend, held back by some missing outer-layer hardening that would be cheap to add. We found no CRITICAL-severity issues; three HIGH-severity ones.

Health scores: Architecture 78/100, Security 64/100, Code Quality 74/100, Performance 72/100, Maintainability 70/100, Production Readiness 61/100.

2. Project Overview

Project-Ticket serves seven types of user accounts: `founder`, `superadmin`, `hr`, `it`, `coordinator`, `employee`, plus five demo-only department roles that aren't wired up to any real backend logic yet (see `App.jsx`, lines 54 to 56). All of them share one dashboard layout. The core areas of the app are: shared HR/IT complaint queues (built from one reusable piece of code), an approvals workflow, leave requests, IT equipment tracking, an HR "desk" module (covering employees, job candidates, interviews, attendance, and job postings), coordinator tasks and projects, and a tracker for production render jobs.

```
Project-Ticket/
├─ .vercel/                the link to the root Vercel project ("project-ticket")
├─ assets/design-tokens.json design settings that generate the frontend's CSS
├─ docs/                  13 markdown files and 5 PDFs; the PRD, TRD, and past
├─ scripts/tokens-to-css.cjs a build helper that turns tokens.json into tokens.css
├─ .tools/jdk-21.../      a bundled Java installer, not referenced by any packa
├─ package-lock.json (root) orphaned: there's no matching root package.json
├─ README.md              just 17 bytes, a placeholder only
└─ main/
    ├─ backend/            the Express API (its own Vercel project and Firebase
    │   ├─ config/         controllers/  middleware/  routes/  utils/
    │   └─ frontend/       the Vite + React 18 website
    │       └─ src/{components,pages,context,hooks,lib,data,styles,utils}/
```

3. Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE	NOTES
Frontend (the website)	React + Vite	React 18.2, Vite 5	The single-page app, with role-based dashboards	A mix of <code>.jsx</code> and <code>.ts</code> files, TypeScript isn't strictly enforced
Routing (navigating between pages)	react-router-dom	6.21	Handles page navigation, with pages loaded only when needed	:
UI Kit (pre-built visual components)	shadcn/ui + Radix + Tailwind	:	Reusable building blocks and styling	:
HTTP client (talks to the backend)	axios	1.6	Makes API calls, one shared connection	Automatically attaches the login token and handles "not logged in" responses
Backend (the server)	Express	^4.18.2	The REST API	No helmet (security headers tool), no morgan (a logging tool)
Login system	Firebase Auth + jsonwebtoken	firebase-admin ^12.0.0, jsonwebtoken ^9.0.2	Stores accounts, issues app-signed login sessions	Password hashing is fully handled by Firebase, using a method called <code>script</code>
Database	Firestore	via firebase-admin ^12.0.0	The main data store, with no fixed schema	Only reachable using full administrative access; no way for the browser to reach it directly
File storage	Firebase Storage	via firebase-admin	HR-desk employee documents	Files are checked and kept in memory (via Multer) before being uploaded, with only certain file types allowed
Email	Nodemailer (SMTP)	^6.9.7	Complaint and notification emails	Text inserted into emails is safely escaped
Hosting	Vercel	:	Two separate projects: frontend and backend	No <code>vercel.json</code> configuration file found for the backend
Automated deployment checks (CI/CD)	none found	:	:	No <code>.github/workflows</code> folder, and no other CI configuration anywhere
Testing	Playwright	end-to-end	Automated browser tests for the frontend	Uses <code>tsc --noEmit</code> for type checking

4. Complete Project Structure

main/backend

- `server.js` : the starting point, sets up the middleware chain (the sequence of checks every request passes through), the error handler, and a graceful shutdown process
- `config/firebase.js` : sets up the connection to Firebase's admin tools, with a fallback to a local test version if real credentials aren't configured
- `controllers/` : one file per subject area; `complaintControllerFactory.js` generates both the HR and IT complaint queues from one shared piece of logic
- `routes/` : 10 files, each connected into `server.js`
- `middleware/` : `authMiddleware` (checks login token and session), `roleMiddleware` (checks whether your role is on the allow-list), `permissionMiddleware` (a finer-grained, database-backed permission check, used only for IT equipment)
- `utils/` : `sessions.js` , `auditLog.js` , `upload.js` (file upload handling via Multer), `mailer.js` , `pagination.js`

main/frontend/src

- `App.jsx` : sets up page routing, wraps everything in shared Context providers, defines which pages load lazily, and gates access through `RequireAuth`
- `context/` : 9 separate shared-data areas (Auth, Leave, Ticket, Approval, Permissions, TaskProject, Render, Asset, HrDesk)
- `utils/api.js` : one shared connection to the backend, plus roughly 60 small helper functions, one per API endpoint
- `components/RequireAuth.jsx` : the page guard, checks you're logged in, checks your role is allowed, and shows a loading skeleton while it figures that out
- `pages/` : one dashboard per role, each loaded only when needed
- `tests/e2e/` : automated browser test scripts

Root-level oddities (worth flagging, not urgent)

- `package-lock.json` at the very top of the project, with no matching `package.json` next to it. We couldn't determine why it exists, it's likely left over from an earlier setup.
 - `.tools/jdk-21.0.12+8/` : a full, bundled Windows Java installer, referenced by nothing in either `package.json` file. We couldn't determine its purpose; it's a safe candidate for removal once confirmed unused.
 - `docs/login-credentials.pdf` : the filename suggests it holds stored login details. We confirmed it isn't tracked by version control, but the fact that it's sitting on disk in a shared documentation folder is still a risk, regardless of its version-control status.
-

5. Architecture

In plain terms: the browser only ever talks to the Express-based backend API; that API is the only thing that ever holds the Firebase administrative credentials, and the only thing that ever touches Firestore, Firebase's login system, or Firebase Storage.

```
Browser (the React website, hosted as the Vercel project "project-ticket")
| Encrypted connection (HTTPS), with a login token attached to every request
v
Backend API (hosted as the Vercel project "backend")
|
|-- trusted-website check (CORS)
|-- reads the incoming request body as JSON
|-- authMiddleware: checks the login token, checks the session hasn't been
|   revoked, checks the account is still active
|-- roleMiddleware / permissionMiddleware: checks your role, and for some
|   pages, checks a more detailed permission too
|-- the page's own logic: does the real work, and checks you own whatever
|   you're changing
|
|   |--> Firestore (all the app's data)
|   |--> Firebase's login system (verifies your identity)
|   |--> Firebase Storage (HR-desk documents)
|   |--> the email server (sends notifications)
|
|-- one central error handler: sends back a safe, generic message, and
    logs the full detail only on the server
```

6. Frontend Architecture

State (the app's shared, currently-active data) lives in plain React Context, one per subject area, rather than a dedicated state-management library. All of it is set up together in `App.jsx`, lines 89 to 97. A comment marked `ponytail:` at `App.jsx`, lines 78 to 86, already documents a known, accepted inefficiency: every Context fetches its data as soon as you log in, regardless of which page you're actually about to visit. The team has flagged this themselves as low priority. Page loading is lazy everywhere (meaning a page's code only downloads when you actually navigate to it) except the login page, and there's a hand-tuned build configuration that groups icon imports and the core React/router code into shared bundles, avoiding more than 40 small separate downloads.

Component to API trace (a representative sample)

COMPONENT (A PIECE OF THE PAGE)	WHICH SHARED DATA AREA IT USES	THE FUNCTION IT CALLS	THE API ADDRESS IT HITS
LoginPage.jsx	AuthContext	api.login()	POST /api/auth/login
HR/IT ticket views	TicketContext	getComplaints() / createComplaint()	/api/hr or /api/it/complaints
Leave pages	LeaveContext	getLeave() / requestLeave()	/api/leave
Approvals views	ApprovalContext	getApprovals() / decide()	/api/approvals
HR Desk module	HrDeskContext	around 15 small helper functions	/api/hr-desk/*

How login works on the browser side

Your login token is stored in either `localStorage` or `sessionStorage` (under the key `fute_token`), the choice depends on whether you checked "remember me" (see `AuthContext.jsx`, lines 38 to 53 and 104 to 129). A piece of code (an "interceptor") automatically attaches the token to every outgoing request as an `Authorization: Bearer <token>` header (see `api.js`, lines 9 to 13). Another interceptor automatically logs you out and redirects you to the login page if any request comes back "not logged in" (`api.js`, lines 21 to 31). The `AuthContext` also re-checks who you are every 15 seconds by calling `getMe()`, so that if the Founder changes your role or permissions while you're already logged in, the change takes effect without you having to log back in.

We found none of the common patterns that allow malicious code to run on a page (`dangerouslySetInnerHTML`, `innerHTML`, or `eval()` anywhere under `src/`). React's default behavior of automatically escaping text is the main defense against that kind of attack (called XSS) here. There's no client-side form-validation library in use (no Zod, Yup, or react-hook-form), form checks are just native HTML5 attributes, with the server doing the real enforcement.

7. Backend Architecture

Here's what happens when the server starts up, in `server.js`: it loads its configuration, then a tool called `express-async-errors` patches the framework so that a failed background operation inside any page's code gets forwarded to the error handler instead of leaving the request hanging forever. Then it immediately refuses to start if the `JWT_SECRET` setting is missing (see `server.js`, lines 16 to 19). Then it connects to Firebase, sets up the trusted-website allow-list, sets up JSON parsing, connects all 10 route files, sets up health-check pages, and finally sets up the central error handler. It shuts down gracefully on a stop signal (giving itself 10 seconds before forcing an exit), except when running on Vercel, where that step is skipped.

What happens to a single request

```
A request comes in
-> authMiddleware      checks the login token, uses a 60-second cache of your
                        profile, checks the session hasn't been revoked, checks
                        the account is active
-> roleMiddleware      checks your role against a route-level allow-list
-> permissionMiddleware a more detailed check (IT equipment pages only, backed
                        by a database-stored permission table)
-> the page's own code  checks the data is valid, checks you own what you're
                        changing, does the actual work
-> Firestore           wrapped in an all-or-nothing transaction wherever the
                        data needs to stay consistent (e.g. a status change
                        plus creating an approval record together)
-> the response         either the successful result, or a safe, generic
                        error via the central handler
```

Password hashing is deliberately absent from this codebase, it's fully handed off to Firebase's login system (there's no bcrypt or argon2 dependency, which is correct given how the system is built, not a gap). The complaint system (both the HR and IT queues) is generated from one shared piece of code, `controllers/complaintControllerFactory.js`, instead of being copy-pasted for each department (lines 262 to 317 hold the ownership and role checks for editing a complaint).

8. Database Architecture

Technology, confirmed: Firestore only, connected via `admin.firestore()` in `config/firebase.js`, lines 44 to 51. Two other database-related packages visible in the installed dependencies (`@supabase` and `@google-cloud`) are only there because `firebase-admin` and `google-auth-library` depend on them internally, they're not used directly by this app's own code. We confirmed this against `package.json`, and `docs/TRD.md` independently notes that the project moved away from an earlier Supabase/Postgres plan and switched to Firebase.

Collections (inferred from where `db.collection(...)` is called in the code; there's no fixed schema and no migration files)

COLLECTION	USED BY
users	authController, superAdminController, departmentController, dashboardController, securityController, authMiddleware
hr_complaints / it_complaints	complaintControllerFactory, dashboardController, analyticsController, slaController
approvals	approvalController, complaintControllerFactory, analyticsController
leave_requests	leaveController, analyticsController
assets, departments, tasks, projects	assetController, departmentController, taskProjectController
settings	systemSettingsController, permissionMiddleware, slaController
sessions, failed_logins, audit_logs	utils/sessions.js, authController, utils/auditLog.js
employees, candidates, attendance, sent_emails	hrDeskController
renders	renderController

There's no database-level "security rules" file anywhere in the project. Because every single database access goes through the backend's full-administrative-access connection (which bypasses those rules by design anyway), this is architecturally consistent rather than an oversight, but it does mean that *all* access control lives inside the Express middleware and controller code, with no backup layer at the database itself. If a direct browser-to-Firestore connection is ever added to the frontend later, it would inherit no rules at all to fall back on. We confirmed no such direct connection exists today.

9. API Surface

Every route requires `authMiddleware` (meaning you must be logged in) unless marked "public" below.

ROUTE FILE	WHAT IT COVERS	WHO'S ALLOWED
authRoutes.js	Registering and logging in (public, rate-limited), fetching your own profile, verifying your password (rate-limited), logging out	:
hrRoutes.js / itRoutes.js	Complaint create/read/update/delete, status updates, field updates, looking up by token	Anyone can create a complaint or view their own; viewing the whole queue and changing status needs hr/it plus founder; editing or deleting needs to be the owner or staff (checked in the code)
approvalRoutes.js	Creating and listing approvals, deciding on one	it/hr/founder can create and view; only founder can decide
coordinatorRoutes.js	Projects, tasks, and updating a task's status	Anyone can read; only coordinator/founder can write; updating a task's status has no role check at all
founderRoutes.js	About 30 pages covering users, analytics, audit logs, service-level agreements, notifications, permissions, settings, departments	Mostly restricted to superadmin; a few "read" pages are open to anyone logged in, but writing is superadmin-only
hrDeskRoutes.js	Employees, candidates, interviews, meetings, attendance, feedback, job postings	hr/founder (plus coordinator can read, and employees can self-serve their own attendance)
leaveRoutes.js	Requesting leave, viewing your own requests, viewing the full queue, deciding	Anyone can request leave and see their own; viewing the queue and deciding needs hr/founder
renderRoutes.js	Viewing, creating, and updating render entries	No role restriction at all, this is intentional according to an inline code comment
securityRoutes.js	Sessions, forced logouts, failed logins, locked accounts, unlocking	Superadmin only

10. Complete Data Flow

Example: filing an HR complaint

```
An employee fills out the complaint form (HrTicketPage)
-> TicketContext.createComplaint() (only basic HTML5 field checks happen here)
-> POST /api/hr/complaints, with the login token attached
-> authMiddleware checks the token, checks the session hasn't been revoked,
    checks the account is active
-> roleMiddleware allows any logged-in role to create a complaint
-> complaintControllerFactory.create() checks required fields, generates a
    6-character tracking token, saves the complaint, links it to the user's
    account
-> mailer.js safely escapes the text and sends a notification email to the
    HR queue
-> the response (including the tracking token) comes back, TicketContext
    updates, and the on-screen list redraws
```

Example: HR resolves and then deletes a complaint: updating the status (restricted to hr/founder) happens inside a Firestore *transaction* that changes the status and creates the linked approval record together, as one all-or-nothing operation, so there's no way to end up in a half-finished state (`complaintControllerFactory.js` , lines 204 to 260). Deleting is restricted to the original owner only, enforced on the server (lines 291 to 317), and also removes the linked approval record as part of the same operation.

11. Authentication

QUESTION	ANSWER	WHERE THIS IS PROVEN IN THE CODE
Where does logging in actually happen?	<code>POST /api/auth/login</code> calls Firebase's Identity Toolkit REST API directly (the Admin SDK can't verify passwords on its own)	authController.js, lines 94 to 112
How are passwords hashed?	Fully handled by Firebase's login system (using scrypt), not implemented anywhere in this codebase	there's no bcrypt or argon2 dependency, which is expected
What kind of token, and how long does it last?	An app-issued login token, valid for 7 days, containing your ID, email, role, name, and session ID	authController.js, lines 55 to 59 and 123 to 127
Is there a refresh mechanism?	No, the long-lived token is instead offset by checking on every request whether the session has been revoked	authMiddleware.js
What happens on logout or revocation?	A session record in Firestore is marked <code>revoked:true</code> , and that's checked on the server on every single request, this is real revocation, not just deleting the token from the browser	authController.js, lines 188 to 193, authMiddleware.js, lines 52 to 53
Protection against repeated password guessing	5 failed attempts locks the account, requiring a superadmin to unlock it; the IP address is logged to <code>failed_logins</code> ; there's also a limit of 10 requests per 15 minutes on login, register, and password-verify	authController.js, lines 10 and 102 to 112; authRoutes.js, lines 12 to 18
What role does self-registration get?	Always hardcoded to <code>employee</code> , a comment in the code confirms this fixed an earlier bug where the role used to be guessed from a pattern in the email address	authController.js, lines 40 to 45
How are privileged accounts created?	Only through a superadmin-restricted page (<code>POST /api/founder/users</code>), using a fixed allow-list of assignable roles that excludes founder and superadmin	superAdminUserController.js, line 8
What's the password policy?	Not enforced by the app itself. Registration only checks the field isn't empty, so Firebase's own default minimum of 6 characters is the real floor. Admin-driven resets do enforce a 10-character minimum, which is inconsistent with the weaker self-registration floor	superAdminUserController.js, lines 268 to 289
Multi-factor authentication (an extra login step)	Unknown, we didn't find any trace of it in the files we reviewed	:

12. Authorization

There are three layers, all enforced on the server: `roleMiddleware` (a route-level allow-list of roles), `permissionMiddleware` (a more detailed, database-backed permission table, currently only applied to creating/editing/deleting IT equipment; superadmin always bypasses it, and anything left unconfigured defaults to allowed), and ownership checks written directly into the controller code for any page that changes data (for example, editing a complaint requires being the owner or staff, deleting requires being the owner, see `complaintControllerFactory.js`, lines 277 to 281 and 297 to 299).

Two things around access control worth a second look:

- The `searchByToken` feature (`complaintControllerFactory.js`, lines 191 to 198) lets any logged-in user, regardless of role, look up *any* complaint using its tracking token, it isn't restricted to the person who owns it. Tokens are 6 random letters and numbers (about 2.2 billion possible combinations), so guessing one isn't realistic in practice, but this is still a deliberate design choice (tokens double as a shared way to check status) rather than a hardened boundary. We think this is likely a minor, intentional case of "insecure direct object reference" (a term for accessing data by guessing its ID), by design.
 - The task-status update page in `coordinatorRoutes.js` has no role check at all, and we couldn't confirm in this pass whether `taskProjectController.js` adds an ownership check underneath it.
-

13. Security Controls Currently Implemented

CONTROL	STATUS	WHERE	HOW STRONG
Ownership and role checks enforced on the server	In place	complaintControllerFactory.js, roleMiddleware, permissionMiddleware	Strong, verified to happen on the server itself, not just hidden in the website's design
Login session revocation	In place	authMiddleware.js and utils/sessions.js	Strong, closes the gap where a 7-day token would otherwise keep working after logout, checked within about a 60-second cache window
Protection against repeated password guessing	In place	authController.js, the failed_logins collection	Strong, combines account lockout, IP logging, and rate limiting
Trusted-website restrictions (CORS)	In place	server.js, lines 32 to 46	Strong, an explicit allow-list, not a wildcard
Safe error messages	In place	server.js, lines 78 to 89	Strong, the client always gets a generic message; full detail only ever goes to the server log
File upload validation	In place	utils/upload.js	Strong, files are kept in memory, only certain file types are allowed, and there's a 10MB cap
Protection against malicious content in emails	In place	utils/mailer.js, lines 14 to 21	Good, any text inserted into an email is safely escaped
Audit logging for admin actions	In place	utils/auditLog.js	Good, creating, editing, deleting a user, changing permissions, and resetting a password are all logged
Keeping secrets out of version control	In place	.gitignore at both the root and backend level; only the example template file is tracked	Confirmed clean by checking exactly which files are tracked
Rate limiting	Partial	Only on authRoutes.js	Weak everywhere else, the rest of the API has no throttling at all
Password policy	Partial	Relies on Firebase's default; the admin-reset floor is inconsistent with self-registration	Weak
Security response headers	Missing	:	No helmet, no Content-Security-Policy, no Strict-Transport-Security, no clickjacking protection, anywhere
Organized, centralized logging	Missing	:	Only console.log/console.error, no dedicated logging library

CONTROL	STATUS	WHERE	HOW STRONG
Automated checks before deploying (CI/CD)	Missing	:	No pipeline exists to run any check at all
Multi-factor authentication	Unknown	:	Not found in the files we reviewed

14. Security Gaps

- No `helmet`, meaning zero default security headers (Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security) exist anywhere in `server.js`. **HIGH**
- **Rate limiting only covers 4 login-related pages**, every `hr/it/leave/coordinator/render/hr-desk` page has no throttling at all. **MEDIUM**
- **No enforced minimum password length at registration**, it relies on Firebase's default 6-character floor, which is inconsistent with the 10-character floor used for admin resets. **MEDIUM**
- The `renderRoutes.js` file and the coordinator's task-status page have no role restriction at all, this looks intentional based on the code comments, but it does widen the potential damage if a low-privilege account is ever compromised. **LOW**
- The `searchByToken` design allows lookups across users by design (see section 12 above). **LOW**
- `docs/login-credentials.pdf` still sits on disk (not tracked by version control, but still a real risk if the documentation folder is ever shared with someone). **MEDIUM**
- With no automated deployment pipeline, there's no automatic dependency-vulnerability scan, code-style check, or test gate before something goes live. **MEDIUM**

15. Sensitive Data Flow

- **Login credentials:** typed into the login form, sent over an encrypted connection to Firebase's Identity Toolkit (this never touches the app's own database), the app never stores a password itself, only Firebase does, and your session afterward is represented by an app-issued login token stored in the browser.
- **Login token:** issued by the server, held in browser storage (which JavaScript on the page can read; there's no `httpOnly` protection here since this app doesn't use cookies), sent as a header on every request, and made invalid either when you log out or when a superadmin force-revokes it, through a Firestore session record.
- **Employee documents (HR Desk):** uploaded through Multer (kept in memory, checked by file type), written to Firebase Storage, and only ever readable through backend-controlled pages restricted to `hr/founder`.
- **Personal information in Firestore:** names, emails, and roles live in the `users` collection; complaint text and attachments live in the complaint collections. We found no field is separately encrypted, protection here relies entirely on controlling who can access the data in the first place.

16. Third-Party Services

SERVICE	PURPOSE	DATA SENT TO IT	HOW IT AUTHENTICATES	RISK LEVEL
Firebase's login system	Stores accounts, verifies identity	Email/password at login, your profile	A service-account certificate (Admin SDK) plus a web API key (REST)	Low, this is a standard, well-managed login setup
Firestore	The main database	All of the app's data	The same service-account certificate	Low, no database-rules file is needed since only the backend's full-access connection ever touches it
Firebase Storage	HR document uploads	PDF/Word/JPG files, capped at 10MB	The same service-account certificate	Low, file type and size are checked first
SMTP (via Nodemailer)	Sends notification emails	Complaint details, with text safely escaped	Username and password stored in environment settings	Low, all output is escaped
Vercel	Hosts both the frontend and the backend	:	A Vercel account / OIDC (a platform-level login mechanism)	:

We found no text messaging, analytics, payment, or mapping integrations anywhere in either the backend or the frontend code.

17. Environment Variables & Secrets

Names only, no values were ever read or displayed.

SETTING NAME	IS IT REQUIRED?	WHERE IT'S USED	IS IT SENSITIVE?
JWT_SECRET	Yes, the server refuses to start without it	server.js, authController	Yes
FIREBASE_PROJECT_ID / CLIENT_EMAIL / PRIVATE_KEY	No, falls back to a local test version if missing	config/firebase.js	Yes
FIREBASE_API_KEY	Yes, needed for real logins to work	authController (talks to Identity Toolkit REST)	Yes
FIREBASE_STORAGE_BUCKET	No, defaults to a standard name based on the project ID	config/firebase.js	No
FIREBASE_AUTH_EMULATOR_HOST / FIRESTORE_EMULATOR_HOST	Development use only	config/firebase.js	No
SMTP_HOST / PORT / USER / PASS	Yes, needed for email to work	utils/mailer.js	Yes
FRONTEND_URL	Yes, needed for trusted-website checks in production	server.js	No
PORT	No	server.js	No
VITE_API_BASE_URL	Yes (used by the frontend)	main/frontend/src/utils/api.js	No
VERCEL_OIDC_TOKEN	Managed automatically by the hosting platform	root .env.local	A platform token, not one of our own service credentials

Secret hygiene, confirmed clean. Checking exactly which files are tracked by version control shows only the two example template files, all real `.env` / `.env.local` files are excluded. We searched the entire codebase for anything that looks like a real key (patterns like `AIza` , `sk_live` , `BEGIN PRIVATE KEY`) and found only two harmless matches: a hardcoded *test* password used only in an automated test, and a placeholder example string shown in the interface. No real secrets exist in the source code.

18. Error Handling

There's one central error handler (`server.js` , lines 78 to 89) that logs the full error detail on the server, and only ever sends back a generic `"Internal server error"` message to whoever made

the request, unless a page's own code deliberately set a specific status. Raw error details and database internals never reach the client, whether the app is running in a test environment or live. A tool called `express-async-errors` makes sure that a failed background operation inside any page's code gets forwarded to this handler, rather than crashing the whole process.

19. Logging & Monitoring

In place: an `audit_logs` collection in Firestore records privileged admin actions (creating, editing, deleting a user; changing permissions; resetting a password) through `utils/auditLog.js`. Failed login attempts are recorded with the visitor's IP address in `failed_logins`.

Missing: there's no dedicated request or error logging library in use, just plain `console.log` / `console.error` calls, which on Vercel land in temporary function logs that aren't easily searched later. We found no integration with any application-monitoring or alerting tool.

20. Performance

- **High impact:** every shared Context fetches its data as soon as you log in, regardless of which page you actually visit (this is a documented, accepted tradeoff, see `App.jsx`, lines 78 to 86). General API pages have no rate limiting, so cost and response time would scale poorly if usage suddenly spiked.
 - **Medium impact:** a 15-second background check (`getMe()`), plus some other similar background checks elsewhere, add ongoing request volume as the number of users grows, this is acceptable at the current scale but worth revisiting before a large increase in headcount.
 - **Low impact:** loading pages only when needed, plus a hand-tuned bundling strategy, already address the biggest risk around a slow initial page load. There are only 5 static images in the whole app, so there's no meaningful gap around lazy-loading images.
-

21. Code Quality

- **Good habit of reusing code:** the HR and IT complaint pages share one piece of code (`complaintControllerFactory.js`) instead of being copy-pasted, a comment in the code explicitly notes this replaced an earlier duplicated pair.
- **Good habit of writing documentation:** the `docs/` folder already contains a product requirements document, a technical requirements document, workflow docs, and two earlier audit reports, all kept reasonably current against the actual code.
- **A mix of JavaScript and TypeScript:** TypeScript checking is configured (`tsc --noEmit`), but most files are still plain `.jsx`, so type-checking coverage is only partial.

- **Uncommitted work in progress** was present in the project folder at the time of this audit (changes to `hrDeskController`, `hrDeskRoutes`, a couple of frontend components, and one brand-new file not yet added to version control), this is a normal, mid-development state, not a problem in itself.
 - **A couple of leftover files:** the root-level `package-lock.json` with no matching manifest, and an unreferenced bundled Java installer under `.tools/`, both are low-cost cleanup candidates.
-

22. Deployment Architecture

```
GitHub (Fute-Services/Project-Ticket repository, main branch)
  |
  v
Vercel (one shared organization account)
|-- Project "project-ticket" -> main/frontend (built with Vite, single-page app)
`-- Project "backend"         -> main/backend   (Express, running as serverless f
      |
      v
      Firebase project (Firestore + login system + file storage)
```

There's no automated deployment-checking pipeline and no Docker setup anywhere in this repository, no `.github/workflows` folder, and no Dockerfile or docker-compose file anywhere in the project. Deployment is most likely handled through Vercel's own automatic "deploy on every Git push" feature, but we couldn't confirm the exact trigger mechanism from the source code alone (there's no `vercel.json` configuration file for the backend project). The encrypted connection (TLS) is handled at Vercel's edge; we found no code in the app itself that forces an HTTPS-only connection, which is consistent with how this type of hosting normally works.

23. Feature Map

FEATURE	FRONTEND PIECE	API	WHO'S ALLOWED	DATABASE COLLECTION
HR / IT Complaints	TicketContext, role-based dashboards	hrRoutes/itRoutes, using complaintControllerFactory	Role-based, plus owner-or-staff for editing/deleting	hr_complaints / it_complaints
Approvals	ApprovalContext	approvalRoutes	it/hr/founder can create; only founder decides	approvals
Leave	LeaveContext	leaveRoutes	Anyone can self-serve; hr/founder decide	leave_requests
IT Assets (equipment)	AssetContext	itRoutes (assets)	Role-based, plus the finer permissionMiddleware check	assets
HR Desk	HrDeskContext	hrDeskRoutes	hr/founder (with some scoped exceptions)	employees, candidates, attendance, sent_emails
Coordinator Tasks/Projects	TaskProjectContext	coordinatorRoutes	Writing needs coordinator/founder; the status-update page has no restriction at all	tasks, projects
Production Renders	RenderContext	renderRoutes	No restriction at all, by design	renders
Admin / Founder console	Founder dashboards	founderRoutes	Mostly superadmin only	users, settings, departments, audit_logs, sla policies
Security console	Founder security views	securityRoutes	Superadmin only	sessions, failed_logins

24. Security Risk Register

(A risk register is a simple table ranking each known issue by how serious it is and what to do about it.)

RISK	SEVERITY	WHERE	RECOMMENDATION
No security response headers	HIGH	server.js	Add the helmet() tool, a one-line change with no behavior risk
General API pages have no rate limiting	HIGH	hr/it/leave/coordinator/render/hr-desk routes	Add a moderate, general-purpose rate limiter across everything, on top of the stricter one that already covers login
docs/login-credentials.pdf still on disk	HIGH	docs/	Remove or move it outside the shared project folder, regardless of its version-control status
No enforced password policy at registration	MEDIUM	authController.js (register/createUser)	Apply the same 10-character (or stronger) floor already used for admin resets
No automated deployment pipeline, so no automatic dependency, code-style, or test check	MEDIUM	project-wide	Add a minimal GitHub Actions workflow that runs a dependency audit, type check, and browser tests on every proposed change
No organized logging or monitoring	MEDIUM	backend	Add a lightweight logging library and send logs somewhere searchable
The renderRoutes file and the task-status page have no role check	LOW	renderRoutes.js, coordinatorRoutes.js	Confirm this is still intentional, and add a role or ownership check if it isn't
The searchByToken lookup works across users	LOW	complaintControllerFactory.js, lines 191 to 198	Acceptable given how many possible tokens there are, but worth documenting as intentional
Leftover root package-lock.json and unused Java installer	LOW	repo root, .tools/	Delete once confirmed unused

25. Trust Boundary Analysis

(A trust boundary is a point where data crosses from a less-trusted zone into a more-trusted one, and each one is worth checking carefully.)

```
You -- your Browser -- the open Internet (encrypted) -- Vercel's edge servers
-- the Express API -- Firebase
```

1. The browser holds your login token in local or session storage, which JavaScript on the page can technically read (there's no httpOnly cookie protection here, since this app doesn't use cookies at all). This is mitigated by the fact that we found no way to inject malicious code into this app's own pages.
2. The connection is encrypted through Vercel's TLS termination; there was no app-level instruction forcing this before (a gap that helmet now closes, see section 24).
3. The trusted-website allow-list restricts which sites' browsers are allowed to call the API at all.
4. Your login token is verified, your session is checked against revocation, and your role and permissions are checked, all before any database access happens.
5. The Firebase administrative credential has full trust and lives only on the server, never reaching the browser. There's no database-level security-rules file, and that's fine only as long as point 4 above stays the one and only way into the database.

26. Security Maturity Score

CATEGORY	SCORE
Login security	80
Permissions	75
Data protection	65
API security	55
Checking incoming data	60
Keeping secrets safe	88
Infrastructure	60
Dependency security	55
Logging and monitoring	40
Error handling	85
Secure configuration	55

Overall: 64 out of 100. This is mainly dragged down by the missing security headers (counted under API security), the lack of organized logging, and the absence of automated dependency scanning (since there's no deployment pipeline). Login security, error handling, and keeping secrets safe are the strongest areas, all backed by real, working mechanisms rather than partial or assumed protections.

27. Final Project Health Score

- **Architecture: 78/100.** A clean separation between the frontend, API, and data layers, consistent reuse of shared code, and a sound request-handling flow. Marked down for having only one layer of defense (the server code) with nothing backing it up at the database level.
 - **Security: 64/100.** See section 26 above. Solid foundations, but missing some outer-layer hardening.
 - **Code Quality: 74/100.** Good structure and a strong habit of writing documentation, marked down for the inconsistent mix of JavaScript and TypeScript and a couple of leftover files.
 - **Performance: 72/100.** Deliberate page-loading and bundling strategies are already in place, marked down for the eager data-fetching (which the team already knows about) and the lack of general rate limiting.
 - **Maintainability: 70/100.** A strong `docs/` habit and shared, reusable code help here; a few controllers weren't read in this particular pass, which keeps confidence at "good, but not fully verified everywhere."
 - **Production Readiness: 61/100.** The core features and access control work well, but no automated deployment pipeline, no security headers, and no monitoring are meaningful gaps to close before scaling much further past current usage.
-

28. Critical Findings

No critical-severity issues were confirmed in this pass. The three HIGH-severity items are all inexpensive to fix: add the `helmet()` tool, extend rate limiting beyond just the login pages, and remove `docs/login-credentials.pdf` from the project folder. None of them require any architectural changes.

29. Recommended Improvements

1. Add the `helmet()` tool to the Express middleware chain, this closes the single biggest gap for the least amount of effort.
2. Apply a general-purpose rate limiter across every page (looser than the one already on the login pages), not just the four login-related endpoints.
3. Enforce a consistent password-length rule at registration, matching the 10-character-or-stronger standard already used for admin resets.
4. Remove `docs/login-credentials.pdf` from the project folder; store any real login details in a password manager instead.
5. Set up a minimal automated pipeline (code-style check, type check, dependency audit, and browser tests) that has to pass before anything merges into the main branch.
6. Add a proper logging library (for example, one called `pino`) and send logs somewhere searchable, so troubleshooting a failed request doesn't depend on temporary Vercel function logs.

7. Revisit whether the `renderRoutes.js` file and the coordinator's task-status page should stay open to everyone as the user base grows.
 8. Clean up the leftover root `package-lock.json` and the unused `.tools/jdk-21.../` folder, once confirmed they really are unused.
 9. Cross-check this report against the existing `docs/PRODUCTION_READINESS_AUDIT.md` and `SECURITY_REMEDIATION_REPORT.pdf` to reconcile anything already tracked or already resolved since 2026-08-24.
-
-

30. Final Conclusion

Project-Ticket is a coherently built internal tool: one API, one data store, consistent server-side enforcement of the access rules that actually matter (ownership, role, and session validity), and a codebase that documents its own tradeoffs honestly rather than hiding them. The gaps found here are the ordinary, low-drama kind: missing headers, thin rate limiting, an inconsistent password policy, and no automated deployment checks. None of them require redesigning anything, and all of them are addressable in a focused week of hardening work rather than a full rebuild. Treat section 29 as the punch list. Nothing in this audit blocks continuing to use the application as-is for its current internal audience.

This was a read-only, static audit, no code, configuration, dependencies, or data were changed to produce this report. Every finding is traced back to a specific file, and a specific line range where applicable. Anything we couldn't verify from the available source code is labeled "unknown" rather than assumed.